# Implementation of FROST Threshold Signatures

## A Working Two-Round Threshold Signing Service in Python


| | |
|---|---|
| Do Thanh Hai | G2607830K |
| Augustine Shoun | G2607353J |
| Somasundaram Kishore | G2609278E |

Nanyang Technological University, Singapore



**Abstract.** A normal digital signature depends on one secret key, so whoever steals that key can sign for the owner. Threshold signatures fix this by splitting the key between several people, so that a fixed number of them must cooperate to sign. In this project we implemented FROST, a threshold signature protocol that needs only two rounds of messages and is specified in RFC 9591. Our Python implementation runs as a small service with a coordinator and three signer nodes. Any two of the three can sign, and the result is an ordinary Ed25519 signature that any standard verifier accepts. We describe the idea behind FROST, how our system is built, how we protect secret values such as nonces, and how we tested it. We also describe the limits of the prototype and what each team member contributed.



**Keywords:** FROST · threshold signatures · Ed25519 · secret sharing · RFC 9591


## 1 Introduction

Digital signatures prove that a message really comes from its owner. They are used everywhere, for example to sign software updates, contracts and bank transfers. The weak point is that a signature depends on a single secret key. If the key is stolen, the thief can sign anything. If the key is lost, the owner can no longer sign at all.

Threshold signing is a way to remove this single point of failure. The secret key is split into several shares, and each share is kept by a different person or machine. A group of n participants is set up so that any t of them can sign together, while fewer than t shares cannot produce a valid group signature under the protocol's security assumptions. For example, with three participants and a threshold of two, one stolen share is not enough to forge a signature, and one lost share does not stop the group from signing.

FROST (Flexible Round-Optimized Schnorr Threshold signatures) is a modern protocol for this purpose. It was introduced by Komlo and Goldberg [1] and later specified in RFC 9591 [2]. Its main strengths are that signing takes only two rounds of messages and that the final result looks exactly like a normal Schnorr signature. With the Ed25519 version, any existing Ed25519 verifier accepts the signature without knowing that a group created it.

We built a Python FROST signing service using the Ed25519 ciphersuite specified in RFC 9591. Our goals were to keep private shares at signer nodes, prevent nonce reuse, and check the resulting signatures with an independent verifier and the RFC test vectors.

## 2 Background

### 2.1 Schnorr Signatures and Ed25519

A Schnorr signature [3] is built from a secret key and a matching public key. To sign, the signer picks a fresh random secret number called a nonce, mixes it with the message and the secret key, and publishes the result together with a public commitment to the nonce. Anyone can check the signature using only the public key. Ed25519 is a widely used signature scheme of this type, and it is defined in RFC 8032 [4].

The nonce is the dangerous part. If the same nonce is ever used for two different messages, the two signatures together reveal the secret key. This is why a large part of our implementation is about handling nonces carefully (Section 4.3).

### 2.2 Splitting a Key with Secret Sharing

Shamir secret sharing [5] is the classic way to split a secret into shares. The idea is easiest to see with a line. Two points are enough to draw exactly one straight line, but one point fits infinitely many lines. If the secret is hidden in the line and each person holds one point, then two people can recover the secret while one person learns nothing. For larger thresholds the same idea works with curves of higher degree.

In FROST, a trusted dealer can create the shares at the start. The dealer also publishes public check values, so that each participant can verify that its share is consistent with the group public key. This is a form of verifiable secret sharing [6]. A different option is distributed key generation, where no single party ever sees the whole key, but we did not implement it (Section 8).

### 2.3 How FROST Signs

Once the shares exist, signing happens in two rounds. In the first round, each chosen signer creates two fresh secret nonces and sends only their public commitments to the coordinator. In the second round, the coordinator fixes the list of participants and sends the message and all commitments to them. Each signer then uses its own share, its nonces and the full commitment list to produce its part of the signature, called a signature share.

The coordinator checks every signature share and combines them into one signature. The key point is that the shares were built so that their sum is a valid signature for the group public key, without anyone ever putting the full secret key back together. Each signature share is bound to the message and commitment list for that signing attempt [1,2].

### 2.4 Security of FROST

RFC 9591 states that FROST resists forgery under the usual model for signature schemes, which is called existential unforgeability under chosen-message attack, and it points to the original analysis [1] and to a later analysis by Bellare et al. [7]. This holds as long as the key shares were handed out securely and an attacker controls at most $t - 1$ signers. The RFC also warns that reused or predictable nonces can leak the key [2].

## 3 System Overview

### 3.1 Components

Our project is a Python 3 application that runs with Docker Compose. Figure 1 shows the parts and Table 1 lists what each part does. The default setting is a 2-of-3 threshold: three signer nodes exist, and any two of them can sign a message together.

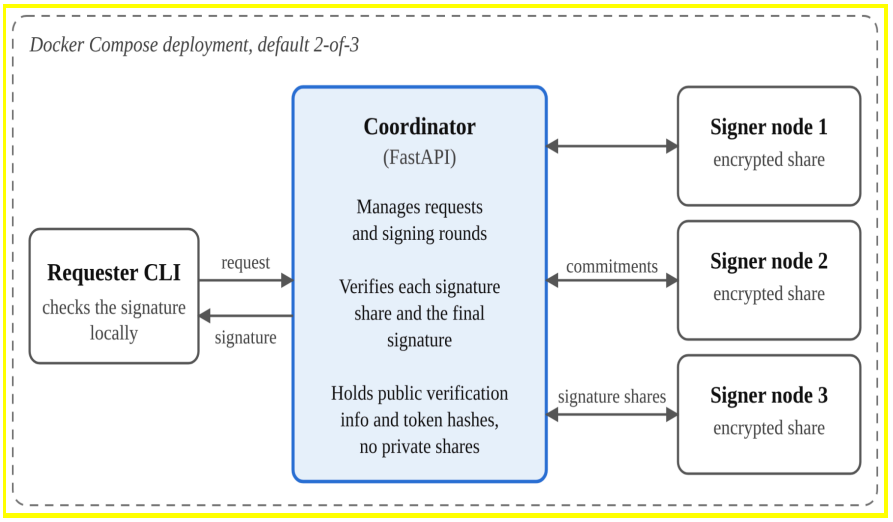



**Fig. 1.** System architecture for the default 2-of-3 deployment. The node links represent protocol exchanges: commitments and signature shares travel to the coordinator; requests and signing packages travel to signers. Private shares are encrypted at rest and used locally by the nodes.

**Table 1.** The main parts of the system.

| Part | What it does |
|---|---|
| Requester client | Sends a signing request and checks the final signature on its own. |
| Coordinator (FastAPI) | Manages requests and the two rounds, checks every signature share and the final signature. It keeps public information only and never holds a private share. |
| Signer node (three) | Keeps one encrypted private share, asks the operator for approval, creates nonces and computes its signature share. |

### 3.2 Libraries and What We Built Ourselves

We used PyNaCl, which wraps the libsodium library [8], for the basic Ed25519 operations on numbers and curve points. We did not write this low-level mathematics ourselves, because reliable implementations already exist. The FROST protocol itself is written by us in Python following RFC 9591: key generation, nonce creation, commitments, the binding of signers to the commitment list, signature shares and aggregation. On top of this we built the coordinator, the signer nodes, the client and the storage and security features described in Section 4. PyCryptodome provides

AES-GCM encryption [9] and is also used in the tests as an independent verifier, so that a signature is never judged only by the code that made it.

## 4 Implementation

### 4.1 Key Generation

At setup, a trusted dealer creates the shares following the dealer procedure of RFC 9591 [2]. Each signer node receives a different share, and public check values let the system confirm that every share belongs to the group public key. Each share is encrypted before it is written to disk. The full group secret is not stored, but the setup step does handle enough secret material to compute it for a short time, so whoever runs the setup must be trusted.

### 4.2 The Signing Flow

Figure 2 shows how a signature is created. A request goes through these steps:

1. The requester sends the message to the coordinator, and the signer nodes ask the operator for approval.
2. Each approving node creates two secret nonces and sends only their public commitments to the coordinator.
3. When enough commitments have arrived, the coordinator fixes the group of signers and sends each of them the message and the commitment list. Each signer computes its signature share.
4. The coordinator checks every share, combines them into one signature and checks that signature against the group public key. The requester then checks it again on its own.

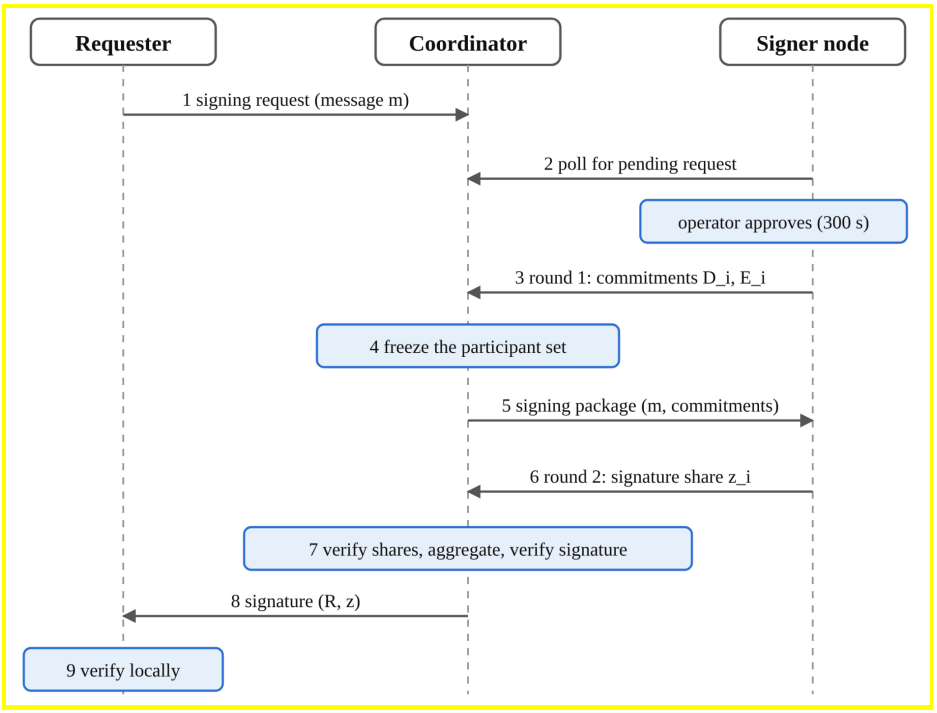



**Fig. 2.** Signing sequence. The signer lane represents each selected signer; 300 s is the operator approval deadline. The coordinator freezes participants and commitments once the threshold is met and collects all shares for that set before aggregation. Each signer consumes its nonces once. Polling responses are omitted for clarity.

Private shares are never sent to the coordinator and are never put back together during signing.

**4.2.1 Signature Share Calculation.** Let S be the selected signer set, G the group generator, Y the group public key, and $s_i$ signer i's secret share. Scalar calculations are modulo the group order q.

In round one, signer i generates fresh secret nonces $d_i$, $e_i$ and publishes:

$$D_i = d_i G, \qquad E_i = e_i G.$$

In round two, each signer checks the signing package, including its own commitments. Following RFC 9591 [2], it derives a binding factor $\rho_i$ from the group public key, message, ordered commitment list and signer identifier. The combined commitment is:

$$R = \sum (D_i + \rho_i E_i), \quad \text{summed over } i \in S.$$

Each signer calculates the challenge $c = H_2(\text{Encode}(R) \,\|\, \text{Encode}(Y) \,\|\, m)$ and its interpolation coefficient:

$$\lambda_i = \prod j / (j - i) \pmod q, \quad \text{product over } j \in S, j \neq i.$$

It then produces:

$$z_i = d_i + \rho_i e_i + \lambda_i s_i c \pmod q.$$

The coordinator checks each share using the public verification share $Y_i = s_i G$:

$$z_i G = D_i + \rho_i E_i + c\lambda_i Y_i.$$

After successful checks, it computes $z = \sum z_i \pmod q$, summed over $i \in S$. The final signature is $(R, z)$, satisfying $zG = R + cY$. Signing combines signature shares without reconstructing the secret key. Each nonce pair must be used only once [2].

### 4.3 Keeping Nonces Safe

Our signer design uses the following rules to prevent nonce reuse. Every request gets new nonces that are generated from secure randomness and the node's private share, as in RFC 9591 [2]. The nonces stay in the memory of the node and are never written to disk. A node marks its nonces as used before it computes the signature share. If the network fails and the share has to be sent again, the node resends the saved share instead of signing a second time. The signer rejects a changed signing package or commitment once it has committed to a signing attempt. The price of this design is that a node which restarts loses its unfinished nonces, so the request has to expire and be started again.

### 4.4 Storage, Approval and Access Control

Private shares are encrypted with AES-256-GCM [9], and the encryption key is derived from a passphrase with Argon2id [10]. Every signing request must be approved by the node operator within five minutes, although the demo can approve automatically. The requester and the nodes use separate random tokens, and the coordinator stores only hashes of these tokens. All inputs such as shares, curve points and signature shares are checked, and limits on request lifetime, request count and message size protect against overload. The containers run without root rights.

## 5 Security Discussion

We assume an attacker who controls the coordinator and fewer signer nodes than the threshold, and who may also steal an encrypted share file. With our 2-of-3 setting this means the coordinator plus one node. As in RFC 9591, we also assume that the dealer is honest and that shares reach the nodes safely. Table 2 lists the main threats, what we did about them and what is still open.

**Table 2.** Main threats, our protections and the remaining limits.

| Threat | Protection | Remaining limit |
|---|---|---|
| Forgery with too few shares | Threshold sharing. A test shows that one node alone cannot sign. | Only holds while fewer than two shares are exposed and the dealer was honest. |
| Dishonest coordinator | It holds no private share and cannot sign without a threshold of signers. Honest signers validate their signing packages; the requester verifies the final signature. | It can refuse or delay service. Share checks performed only by the coordinator cannot be trusted if it is malicious. |
| Reused nonce | Fresh nonces, marked as used before signing, saved share resent on retry. | A node restart forces a new request. |
| Stolen share file | AES-256-GCM encryption with an Argon2id key. | A weak passphrase can be guessed. The demo shares one passphrase. |
| Bad input or overload | Input checks, expiry times and size limits. | No independent security audit. |
| Unauthorized access | Separate tokens, stored only as hashes. | The demo uses plain HTTP inside its own network. |

The security of the signatures themselves rests on the published proofs for FROST [1,2,7]. Our tests give evidence that our code behaves correctly, but they are not a security proof, and the prototype has not been audited.

# 6 Results

## 6.1 Tests

The report records two committed automated tests, both reported as passing, alongside additional development checks listed in Table 3. Figure 3 shows the terminal output of the two committed tests. The committed tests use PyCryptodome to verify the generated signatures independently. The final submission should include the test commands and supporting scripts for the results reported here.

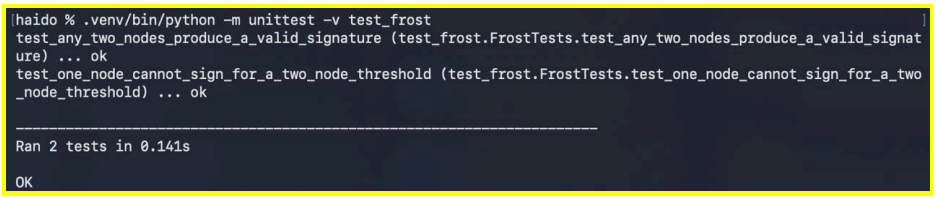



**Fig. 3.** Output of the committed unit tests (test_frost): both tests pass, covering valid signatures from any two nodes and failure with a single node.

**Table 3.** Test results. “Committed” tests are in the repository, “development” checks were run while building the system.

| Check | What it shows | Result |
|---|---|---|
| Committed: every pair of signers | Each pair of the three nodes produces a signature that PyCryptodome accepts. A changed message is rejected. | Pass |
| Committed: single node | One node alone cannot sign when two are needed. | Pass |
| Development: bad input | Malformed input, corrupted shares, mismatched keys and invalid curve points are handled. | Pass |
| Development: Docker run | Setup and signing work with Docker on the tested Windows setup (Section 6.2). | Pass |

### 6.2 Demo Run

To check the whole system and not only single parts, we ran the demo from the README on a Windows computer with Git Bash and Docker. The script run_demo.sh starts the coordinator and three nodes and signs the message “Hello” with two of them. We then sent a second message with the requester command and stopped the project. Table 4 lists what we saw. Figures 4 to 7 show the matching terminal output. In both signing requests the final signature was reported as verified. This shows that the parts work together in our 2-of-3 setting. Like the tests in Section 6.1, it is not a proof of protocol correctness.

**Table 4.** Observed results of the demo run.

| Step | What we saw |
|---|---|
| Start with ./run_demo.sh | The image was built and four containers (the coordinator and three nodes) reported Healthy. Only the coordinator is reachable from the host, on 127.0.0.1:8080. |

| Step | What we saw |
|---|---|
| Request 1: message “Hello” | Approved automatically. The result showed status “completed” and verified: true. |
| Request 2: message “Another message” | Sent with the requester command from the README. The result again showed status “completed” and verified: true, for the same vault. |
| Stop with docker compose down | The three nodes, the coordinator and the network were removed. Vault data stays in the .vault folder. |

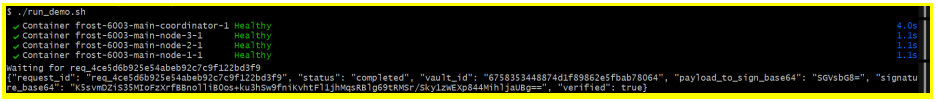



**Fig. 4.** Start of the demo with ./run_demo.sh: the coordinator and three nodes are healthy, and the first request (“Hello”) is completed and verified.




**Fig. 5.** docker ps listing the three signer nodes and the coordinator; only the coordinator is published on 127.0.0.1:8080.

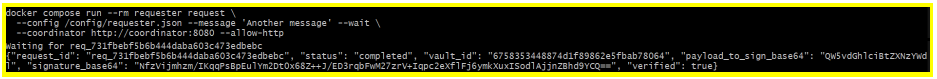



**Fig. 6.** Second signing request (“Another message”) sent with the requester command; the result is completed and verified.

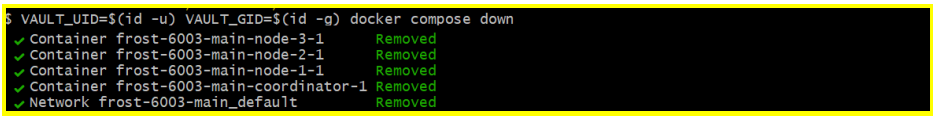



**Fig. 7.** docker compose down removing the three nodes, the coordinator and the network.

### 6.3 What We Did Not Measure

We did not measure performance or scaling, and our service demonstration used the 2-of-3 setting. Two further successful signing sessions were also run on Windows with Docker Desktop, using the messages “Hello” and a second test message. These

runs demonstrate integration on that setup; they do not establish complete protocol correctness.

# 7 Team Contributions and Challenges

## 7.1 Who Did What

The work was shared across the team. Table 5 lists the main areas of work in the project.

**Table 5.** Main areas of work in the project.

| Names | Area | Work |
|---|---|---|
| Do Thanh Hai | Cryptography and coordinator | Core FROST cryptography following RFC 9591, the coordinator, the final signing integration and the combined Docker build. |
| Augustine Shoun | Signer tool | Signer command-line tool: setup, encrypted share storage, polling the coordinator, operator approval with a deadline, terminal output, non-root signer container, and testing and fixes on Windows. Unit tests for the signer tool were also written during development. |
| Somasundaram Kishore | Documentation | Prepared the project report, explained the system design and implementation, created diagrams and figures. |

## 7.2 Challenges

**Nonce safety.** Handling retries without reusing nonces was a main implementation challenge. Section 4.3 describes the approach.

**Integration.** The signer tool was initially developed against a mock coordinator and later connected to the cryptographic core and real coordinator. The final signing design exchanges commitments and signature shares, without sending private key shares to the coordinator.

**Checking correctness.** Development checks included comparisons with the RFC 9591 test vectors. These checks should be retained with the submitted source so that the results can be reproduced.

**Windows.** Running the demo on Windows caused three problems, shown in Table 6. The permission fix only changes how the containers see the file and does not protect the Windows host itself.

**Table 6.** Windows problems and the fixes we tested.

| Problem | Fix |
|---|---|
| The passphrase file looked too open to the signer tool (mode 0o777). | A Windows compose file copies the secret to a private temporary file inside the container. |
| Git Bash changed the container path in the requester command. | Set MSYS_NO_PATHCONV=1 before the command. |
| The demo script failed because of Windows line endings. | Convert the script from CRLF to LF. |

## 8 Limitations and Conclusion

Our prototype has clear limits. Key generation uses a trusted dealer instead of distributed key generation. The coordinator is a single point for availability, and a coordinator restart loses its request state. A signer restart loses that signer's unfinished nonces. The Docker demo runs all nodes on one machine with a shared passphrase, so it does not give the separation of real independent operators. Python cannot guarantee that secrets are erased from memory. We did not implement key rotation, share refresh or participant replacement, and the project has not been audited. These items would be the natural next steps.

We built a Python service for 2-of-3 FROST signing with the Ed25519 ciphersuite. The reported results include independent signature verification, rejection of a changed message, a working Docker demo run, and development checks against RFC test vectors. The main remaining limitations are trusted-dealer setup, availability through a single coordinator, and the lack of an independent security audit.

## Tool Use Declaration

Claude (Anthropic) assisted with the original report draft and figure-generation script. ChatGPT (OpenAI) assisted with reviewing and simplifying the report. Codex (OpenAI) assisted with coding.

## References

[1] Komlo, C., Goldberg, I.: FROST: Flexible Round-Optimized Schnorr Threshold Signatures. In: Dunkelman, O., Jacobson Jr., M.J., O'Flynn, C. (eds.) SAC 2020. LNCS, vol. 12804, pp. 34–65. Springer (2021)

[2] Connolly, D., Komlo, C., Goldberg, I., Wood, C.A.: The Flexible Round-Optimized Schnorr Threshold (FROST) Protocol for Two-Round Schnorr Signatures. RFC 9591 (June 2024). https://www.rfc-editor.org/rfc/rfc9591.html

[3] Schnorr, C.P.: Efficient signature generation by smart cards. Journal of Cryptology 4(3), 161–174 (1991)

[4] Josefsson, S., Liusvaara, I.: Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032 (January 2017). https://www.rfc-editor.org/rfc/rfc8032.html

[5] Shamir, A.: How to share a secret. Communications of the ACM 22(11), 612–613 (1979)

[6] Feldman, P.: A practical scheme for non-interactive verifiable secret sharing. In: 28th Annual Symposium on Foundations of Computer Science (FOCS), pp. 427–438. IEEE (1987)

[7] Bellare, M., Crites, E., Komlo, C., Maller, M., Tessaro, S., Zhu, C.: Better than advertised security for non-interactive threshold signatures. In: Dodis, Y., Shrimpton, T. (eds.) CRYPTO 2022. LNCS, vol. 13510, pp. 517–550. Springer (2022)

[8] libsodium and PyNaCl documentation. https://doc.libsodium.org and https://pynacl.readthedocs.io

[9] Dworkin, M.: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. NIST Special Publication 800-38D (2007). https://doi.org/10.6028/NIST.SP.800-38D

[10] Biryukov, A., Dinu, D., Khovratovich, D., Josefsson, S.: Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications. RFC 9106 (September 2021). https://www.rfc-editor.org/rfc/rfc9106.html

Project Repository Link: https://github.com/HarryDo15/FROST-6003